

Proyecto T2

Auditoría de Seguridad para Desarrolladores: Sesión de Trabajo

Nombre de los alumnos:

Ignacio Bobes Elso
Pablo Soriano Martinez
Ricardo Crespo Berg
Guillermo Torres del Peso

Curso académico:

1ºDAM Grupo2

Tutora/Tutor del proyecto:

Carmelo Escribano Lopez

Entregable 1: Acta de Constitución y Arranque del Proyecto

1. Identificación del Proyecto y Composición del Grupo

El proyecto, titulado "**Auditoría de Seguridad para Desarrolladores**", se establece como una iniciativa de trabajo orientada al análisis y la mejora de la seguridad en entornos donde se publica software. El grupo de trabajo está compuesto por los alumnos **Ignacio Bobes**, **Pablo Soriano**, **Ricardo Crespo** y **Guillermo Torres**. La sesión inicial ha servido para asentar una idea común entre los miembros del equipo y dejar claros los compromisos técnicos necesarios para poner en marcha el laboratorio de seguridad.

2. Objetivo Técnico y Alcance

El propósito fundamental de este trabajo consiste en la creación de un entorno controlado que funcione como un laboratorio para comprobar qué tan segura es una aplicación web mediante el uso de técnicas de ataque y defensa. Se pretende montar una estructura de servicios de nivel profesional que permita recrear situaciones reales de riesgo. Para lograrlo, se ha decidido utilizar un sistema basado en contenedores que facilite ver de cerca los fallos de seguridad y aplicar soluciones prácticas, como el cifrado de la información y la creación de procesos automáticos para las copias de seguridad.

3. Decisiones Técnicas y Definición del Entorno

Durante la reunión de arranque, se han tomado decisiones importantes sobre el conjunto de herramientas que se van a utilizar. Se ha elegido montar un gestor de contenidos basado en WordPress, que funcionará conectado a una base de datos MySQL. Para asegurar que los servicios estén bien separados y simular una red profesional, se ha acordado el uso de redes internas de Docker, lo que permitirá que el tráfico de la web y el de la base de datos no se mezclen sin control.

Al mismo tiempo, se ha definido un entorno de trabajo donde conviven equipos con Windows 11 y Ubuntu, lo que obliga a llevar un control muy riguroso de los archivos. Por este motivo, se usará GitHub como lugar central para guardar el código, estableciendo un sistema de ramas para que el trabajo de cada uno no interfiera con el del resto. En cuanto a la protección del equipo principal, se ha decidido configurar un muro de fuego mediante UFW para vigilar quién accede a los servicios mientras se realizan las pruebas de seguridad.

4. Organización Técnica y Reparto de Tareas

La organización del grupo se ha diseñado para aprovechar mejor las habilidades de cada integrante según los roles que ya se habían planteado. **Ignacio**, como responsable del equipo, se encargará de organizar los pasos a seguir y vigilar que se cumplan las fechas importantes. **Guillermo** será el encargado de gestionar el repositorio en GitHub, preparando la estructura de las carpetas y controlando cómo se unen los cambios realizados por los demás.

Por otro lado, **Ricardo** asumirá la tarea de buscar la documentación necesaria y redactar los textos del proyecto para que toda la actividad quede bien explicada.

Pablo dedicará sus primeros esfuerzos a investigar formas de poner a prueba el sistema y a preparar las herramientas de análisis en Kali Linux. Para estar en contacto, el equipo usará WhatsApp para avisos rápidos y Discord para las reuniones de trabajo técnico. Se han reservado las tardes de los lunes y jueves, que son los días con clase de sistemas, para conectarse y avanzar juntos en el proyecto.

5. Análisis de Riesgos y Problemas Identificados

A pesar de tener un plan claro, se han detectado algunos riesgos que podrían dificultar el avance. El hecho de trabajar con sistemas operativos distintos en el grupo es un reto a la hora de unificar los guiones de instrucciones en Bash, ya que podrían aparecer fallos de compatibilidad. También existe el riesgo de que al probar ataques de fuerza bruta con herramientas como Hydra, el contenedor de WordPress se bloquee por falta de memoria o potencia si no se ajustan bien los límites. Por último, al no tener todavía una dirección de internet fija para el repositorio, los avances se guardarán de forma local hasta que el espacio común esté totalmente listo.

Entregable 2: Diseño inicial de la arquitectura

1. Modelo de arquitectura y entorno de ejecución

El diseño del sistema se fundamenta en un modelo de arquitectura basado en contenedores, utilizando la tecnología de virtualización ligera proporcionada por Docker. Esta elección permite separar los servicios de forma que cada componente funcione de manera aislada, facilitando tanto el despliegue como la futura auditoría de seguridad. Para la gestión y puesta en marcha de todo el entorno, se empleará la herramienta Docker Compose, la cual permite definir en un único archivo de configuración todos los servicios, redes y volúmenes necesarios para que el laboratorio sea reproducible en los diferentes equipos del grupo.

2. Componentes principales y servicios

Se ha definido una estructura de dos capas diferenciadas para organizar los servicios principales del proyecto. La primera capa corresponde a la aplicación web, para la cual se utilizará una imagen oficial de WordPress que actuará como el punto de entrada para los usuarios y, por tanto, como el objetivo principal de las pruebas de seguridad. La segunda capa consiste en el motor de base de datos MySQL, encargado de almacenar toda la información técnica y de configuración del sitio web. Para asegurar que los datos no se pierdan al detener los contenedores, se han configurado volúmenes de persistencia que vinculan carpetas del equipo anfitrión con los datos críticos del servidor y la base de datos.

3. Diseño de red y aislamiento

Uno de los puntos clave de este diseño es la creación de redes virtuales internas mediante Docker Networks para controlar el flujo de comunicación entre los servicios. Se ha proyectado la creación de una red privada para la base de datos, de modo que el contenedor de MySQL no sea accesible desde el exterior, permitiendo únicamente la conexión directa desde el contenedor de WordPress. Este esquema mejora la seguridad del sistema al reducir la superficie de ataque, obligando a cualquier intento de intrusión a pasar primero por la capa de aplicación. El acceso externo para la administración del sistema se realizará de forma protegida mediante el protocolo SSH, utilizando un sistema de claves públicas para evitar el uso de contraseñas vulnerables.

4. Integración del entorno de auditoría

Para llevar a cabo las tareas de seguridad activa, el diseño incluye la integración de un sistema Kali Linux que actuará como la estación de trabajo desde la cual se lanzarán los análisis. Esta herramienta permitirá realizar escaneos de puertos mediante Nmap para identificar servicios visibles y monitorizar el tráfico de red con tcpdump para entender cómo viajan los datos entre los componentes. La relación entre el sistema de auditoría y los contenedores se basará en una conexión de red directa que permita simular un entorno de red de área local (LAN), donde el atacante intenta descubrir debilidades en la configuración de los servicios web expuestos.

5. Seguridad del host y medidas de protección

La protección de los equipos físicos que ejecutan los contenedores se reforzará mediante la configuración de un cortafuegos UFW, encargado de limitar estrictamente qué puertos están abiertos al exterior. Además de la seguridad en la red, se ha planificado la implementación de medidas de criptografía para proteger la información sensible almacenada. Se utilizará un sistema híbrido que combine el cifrado asimétrico para el intercambio seguro de claves y el cifrado simétrico para asegurar los datos de los volúmenes, garantizando que, incluso en caso de una brecha en el servidor, la información permanezca ilegible para personas no autorizadas. Finalmente, se integrarán procesos automáticos para realizar copias de seguridad de los volúmenes de Docker, cuya validez será comprobada mediante el uso de funciones de resumen para asegurar que los respaldos no hayan sido alterados. Respecto a la organización del código y el control de versiones, el espacio de trabajo en GitHub se encuentra actualmente en fase de definición, por lo que se establecerán las rutas y la estructura de carpetas definitiva una vez que el repositorio común esté totalmente operativo y accesible para todos los miembros del equipo.

Entregable 3: Entorno y herramientas de trabajo

1. Sistemas operativos y plataforma de virtualización

El desarrollo de este laboratorio se llevará a cabo en un entorno que combina diferentes sistemas operativos para cubrir todas las necesidades del proyecto. Las estaciones de trabajo principales utilizarán **Windows 11** y **Ubuntu**, lo que permite probar la compatibilidad de las herramientas en distintos entornos de escritorio. Como base para la ejecución de los servicios, se ha seleccionado **Docker Desktop** junto con **Docker Compose**, lo que facilita la creación de contenedores aislados y ligeros sin la necesidad de instalar cada servicio de forma tradicional en el equipo anfitrión. Esta configuración asegura que el entorno de WordPress y la base de datos MySQL funcionen de la misma manera en todos los ordenadores del grupo.

2. Software de auditoría y análisis de red

Para las tareas de seguridad activa, el equipo contará con una Máquina Virtual (MV) de Kali Linux, que servirá como centro de mando para realizar las pruebas de penetración y el análisis de debilidades sin interferir con el sistema operativo principal. Entre las herramientas instaladas en este entorno se encuentra Nmap, que se utilizará para descubrir qué puertos y servicios están abiertos en el servidor web. También se empleará Hydra para realizar pruebas controladas sobre la robustez de las contraseñas en el sistema de acceso de la aplicación. Para complementar este análisis, se usará Wireshark, una herramienta que permite capturar y ver de forma gráfica el tráfico de datos que circula por las redes virtuales, lo que ayudará a detectar posibles fugas de información o protocolos que no estén bien protegidos.

3. Herramientas de desarrollo y automatización

La creación de los guiones de instrucciones y la gestión del entorno se apoyará en el uso de Cursor, un editor de código que utiliza inteligencia artificial para ayudar a generar scripts de mantenimiento de forma más rápida. El lenguaje principal para la automatización de tareas en el sistema será Bash, que se usará para crear herramientas que realicen copias de seguridad de forma automática y verifiquen la integridad de los archivos. Además, para asegurar que la administración de los servidores se realice sin riesgos, se configurará el acceso remoto a través de SSH. Para este acceso, se utilizará un sistema de claves basado en el algoritmo RSA-4096, lo que garantiza un nivel de seguridad muy alto al evitar el uso de contraseñas convencionales que podrían ser adivinadas.

4. Seguridad de los datos y control del código

En el apartado de protección de la información, se ha decidido utilizar herramientas como **OpenSSL** o **GPG**, que permitirán aplicar algoritmos de cifrado modernos para proteger los datos más sensibles del proyecto. En concreto, se utilizará el estándar **AES-256** para asegurar que la información guardada en los discos sea privada y resistente a intentos de lectura no autorizados. Esto garantiza que la información de los usuarios y la configuración interna del servidor estén protegidas bajo estándares profesionales. Para el control de todo el trabajo realizado, se empleará **Git** como sistema de gestión de versiones, permitiendo que el equipo trabaje de forma conjunta en diferentes partes del proyecto sin borrar los avances de los demás. La estructura inicial del repositorio se organizará en carpetas separadas para los archivos de Docker, los guiones de automatización y la documentación técnica generada.